Lezione 12 - Collection

Introduzione alle Collection in Java

Nelle lezioni precedenti abbiamo rivisto i concetti fondamentali di **oggetti e classi**, imparando a creare metodi, costruttori e attributi, e a gestire le **eccezioni** tramite `try-catch`, `throws`, `finally`.
Abbiamo anche visto come confrontare oggetti con il metodo `equals()`, e come utilizzare le **liste dinamiche** come `ArrayList` per memorizzare e manipolare insiemi di oggetti in modo più flessibile rispetto agli array tradizionali.

In questa lezione introduciamo le **Collection**, ovvero le strutture dati di Java progettate per contenere e gestire **insiemi di oggetti** in maniera efficiente. Analizzeremo:

- L'interfaccia **Collection** come base comune di tutte le collezioni
- Le principali sotto-interfacce, in particolare **List**, e le loro implementazioni concrete: `ArrayList`, `LinkedList` e `Vector`
- I **metodi principali** per inserire, leggere, cercare e rimuovere elementi
- L'uso di **Iterator** e del **costrutto foreach** per navigare tra gli elementi
- L'importanza dei **generics**, che consentono di definire il tipo di oggetti contenuti in una collezione e riducono la necessità di cast
- Le problematiche legate ai **tipi primitivi** e la soluzione offerta da **autoboxing** e **unboxing**

> In pratica, passeremo dalla gestione manuale degli array a una gestione più flessibile, tipizzata e orientata agli oggetti, grazie alle strutture della gerarchia delle Collection.

Collection (Java)

In Java, `Collection` è un'interfaccia del package `java.util` **che modella un insieme di oggetti.**

Rappresenta la **radice (root) della gerarchia delle strutture dati dinamiche:**

- **cioè il punto di partenza comune per tutte le principali collezioni del linguaggio.**

Quando si parla di Collection, non si fa riferimento a una struttura dati specifica (come un array o una lista), **ma a un contratto generale che definisce cosa significa "gestire un insieme di elementi".**

> L'interfaccia `Collection` **stabilisce quali operazioni di base devono essere disponibili in qualunque struttura dati che contenga oggetti.**

Da questa interfaccia derivano altre sotto-interfacce più specializzate, come:

- `List`
- `Set`
- `Queue`

Ognuna di queste aggiunge regole specifiche sul comportamento degli elementi (ordine, duplicati, modalità di accesso, ecc.).

Caratteristiche fondamentali di Collection

- Le Collection hanno alcune proprietà comuni importanti:

1. Gestione esclusiva di oggetti

Le Collection accettano e gestiscono solo oggetti, NON tipi primitivi.

Questo significa che non possiamo inserire direttamente un `int`, un `double` o un `char`. Tuttavia, **Java fornisce i wrapper class (`Integer`, `Double`, `Character`, ecc.) e grazie al meccanismo di autoboxing la conversione avviene automaticamente.**

Esempio:

Java

```java
Collection<Integer> numeri = new ArrayList<>();
numeri.add(5);   // il valore 5 viene automaticamente convertito in Integer
```

2. Operazioni generiche comuni

L'interfaccia `Collection` definisce una serie di **operazioni di base**, valide per qualunque tipo di collezione:

- **aggiungere un elemento**

- **rimuovere un elemento**
- **verificare la presenza di un elemento**
- **conoscere il numero totale di elementi**
- **verificare se la collezione è vuota**
- **ottenere un oggetto per la navigazione sequenziale (`Iterator`)**

Questi metodi non dipendono dalla struttura concreta (lista, insieme, coda…), ma rappresentano il **minimo comune denominatore** di tutte le Collection.

3. Non esiste un'implementazione diretta

Un punto molto importante:

- **`Collection` è un'interfaccia, quindi non può essere istanziata direttamente.**

Non possiamo scrivere:

```java
Collection<String> c = new Collection<>(); // ERRORE
```

Esistono invece:

- **sotto-interfacce (`List`, `Set`, `Queue`)**
- **classi concrete che implementano tali interfacce (es. `ArrayList`, `LinkedList`, `HashSet`, ecc.)**

Questo segue il principio della **programmazione contro interfaccia**:

- **si dichiara il tipo più generale possibile (`Collection` o `List`), ma si istanzia una classe concreta.**

4. Duplicati: dipende dall'implementazione

Non tutte le Collection si comportano allo stesso modo:

- **Alcune permettono elementi duplicati (es. le `List`)**
- **Altre non consentono duplicati (es. i `Set`)**

Questa differenza non è definita direttamente da `Collection`, ma dalle sue sotto-interfacce.

Iterator

Per poter accedere agli elementi di una Collection in modo sequenziale, Java utilizza **l'interfaccia** `Iterator`, anch'essa nel package `java.util`.

> **Nota**
>
> L'interfaccia Iterator è un interfaccia a se di rispetto all'interfaccia Collection

L' `Iterator` è uno strumento che:

- **consente di navigare gli elementi senza conoscere la struttura interna della collezione.**

Questo è un concetto fondamentale:

- **non importa se la Collection è una lista, un insieme o una coda — l'iteratore fornisce un modo uniforme per attraversarne gli elementi.**

Ottenere un Iterator

Ogni Collection mette a disposizione il metodo:

Java

```java
iterator()
```

che restituisce un oggetto `Iterator`.

Esempio:

Java

```java
Collection<String> nomi = new ArrayList<>();
Iterator<String> it = nomi.iterator();
```

Metodi principali di Iterator

L'interfaccia `Iterator` definisce tre metodi fondamentali:

1. `boolean hasNext()`

- **Verifica se esiste un elemento successivo.**

- **Serve per controllare se l'iterazione può continuare.**

2. `Object next()`

- **Restituisce l'elemento successivo e sposta l'iteratore in avanti.**
- **È importante notare che `next()` modifica la posizione interna dell'iteratore.**

3. `void remove()`

- **Rimuove l'ultimo elemento restituito da `next()`.**

Ciclo tipico con Iterator

L'utilizzo standard è il seguente:

Java

```java
Iterator<String> it = nomi.iterator();
while (it.hasNext()) {
    String nome = it.next();
    System.out.println(nome);
}
```

Il ciclo continua finché esistono elementi.

ⓘ Nota importante sull'uso

Un `Iterator` può essere utilizzato una sola volta per percorrere la Collection.

Una volta terminata l'iterazione:

- **non si può "tornare indietro"**
- **non si può riutilizzare lo stesso Iterator**

Per effettuare una nuova scansione è necessario chiamare nuovamente `iterator()`.

🔗 Collegamento concettuale

L'Iterator è un esempio del principio di **separazione tra struttura e navigazione**:

- **La Collection si occupa di contenere e organizzare gli elementi.**
- **L'Iterator si occupa di attraversarli.**

Questo rende il codice più flessibile e indipendente dall'implementazione concreta.

`Iterato` vs Loop

Abbiamo detto che `Iterator` è un interfaccia separate e che serve ESCLUSIVAMENTE per navigare una Collection
Perché allora non basta un semplice `for`?
Qui la differenza è sostanziale:

- un ciclo `for` classico funziona bene **solo quando hai accesso tramite indice, ad esempio con un array o con una `List`:**

Java

```java
for (int i = 0; i < lista.size(); i++) {
    System.out.println(lista.get(i));
}

```

Questo funziona perché:

- le liste hanno un ordine definito
- esiste il concetto di indice
- esiste il metodo `get(i)`
  Ma ora pensiamo ad un `Set`.
  I `Set` in Java:
- **non garantiscono l' ordine degli elementi**
- **non hanno indici**
- **e quindi non hanno un metodo `get(i)`**

Quindi un `for` con indice non è applicabile a tutte le Collection.

Di conseguenza l' `Iterator` serve a fornire un meccanismo uniforme di attraversamento indipendente dal tipo concreto di Collection.
In altre parole:

> **Non importa se si sta usando una `List`, un `Set` o una `Queue`:**

- **l'Iterator ti permette di scorrerli tutti allo stesso modo.**
  Questo è un esempio di **programmazione contro interfaccia.**
  Quindi si può scrivere codice generico:

Java

```java
public void stampa(Collection<String> c) {
  Iterator<String> it = c.iterator();
  while (it.hasNext()) {
      System.out.println(it.next());
  }
}

```

Quindi l'uso dell' `Iterator` garantisce lo scorrimento e la navigazione su qualsiasi tipo concreto di collection(`ArrayList` , `LinkedSet` , `HashSet` , ecc.), senza sapere quale implementazione concreta sta utilizzando la Collection in questione.
E il ciclo `while` ?
Il `while` è solo una struttura di controllo.
Non permette di sapere **come ottenere l'elemento successivo** da una struttura dati generica.
L' `Iterator` invece:

- **incapsula lo stato della posizione corrente**
- **gestisce internamente il modo in cui si passa all'elemento successivo**
- **nasconde la struttura interna della Collection**

E allora se si usasse un `for-each` per navigare lungo una qualsiasi collection?
La risposta è si, si può fare!
MA:
quando si scrive

Java

```java
for (String s : collection) {
   System.out.println(s);
}

```

In realtà si sta usando internamente un Iterator.
Il compilatore lo trasforma in qualcosa di equivalente a:

```java
Iterator<String> it = collection.iterator();
while (it.hasNext()) {
  String s = it.next();
  System.out.println(s);
}

```

Quindi il `for-each` **è solo una sintassi più compatta sopra l'iterator**.
Un altro motivo fondamentale è:

- **L'Iterator permette di rimuovere elementi durante l'iterazione in modo sicuro.**

```java
Iterator<String> it = collection.iterator();
while (it.hasNext()) {
   if (it.next().equals("test")) {
       it.remove();
   }
}

```

Se provassi a rimuovere elementi con un `for` tradizionale mentre stai iterando, potresti ottenere una `ConcurrentModificationException`.
Questo perché il `for-each` è pensato, in Java, per iterazioni di sola lettura o comunque senza modifiche strutturali.
Quindi se si devono:

- **rimuovere elementi**
- **filtrare elementi**
- **fare modifiche durante l'attraversamento**
  Allora l'Iterator è lo strumento corretto

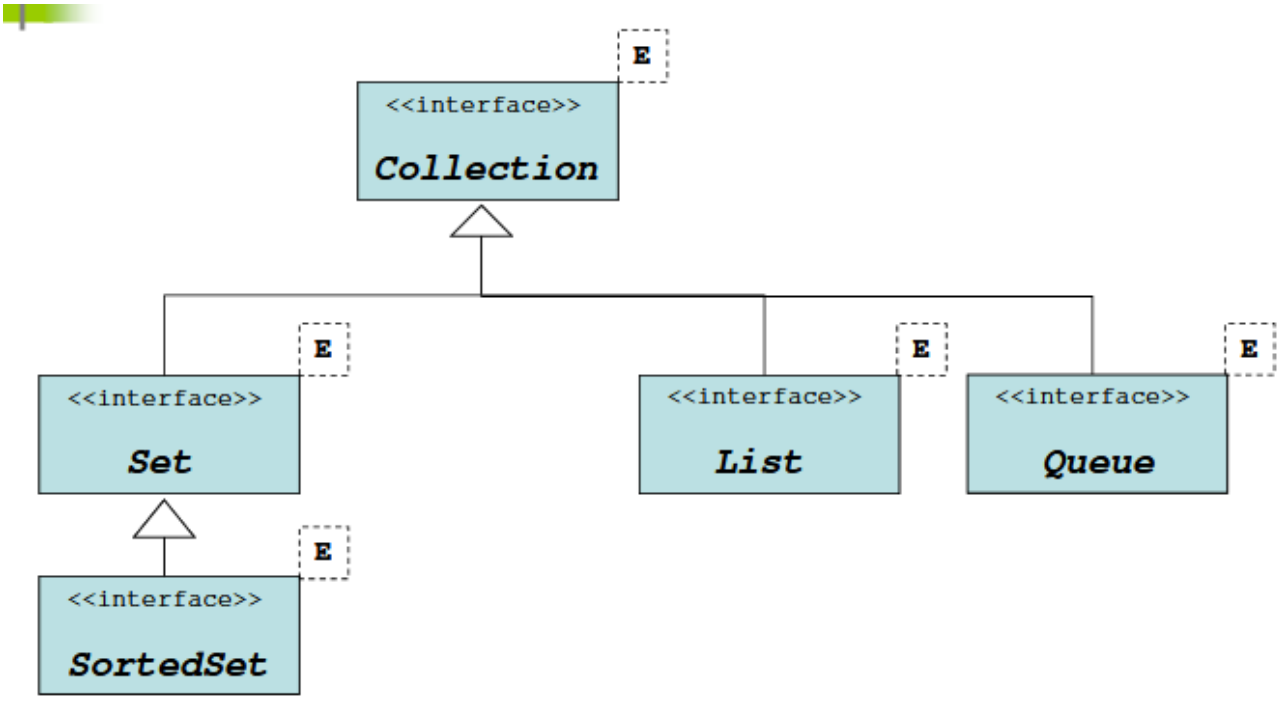
Architettura delle Collection

Per comprendere meglio cosa sono le Collection e come sono organizzate, è utile osservare la **gerarchia delle interfacce** che compongono il framework.

Come si può notare dallo schema, l'interfaccia `Collection`:

- **rappresenta il punto di partenza comune (root) della gerarchia.**

Da essa derivano diverse sotto-interfacce più specifiche, ognuna con caratteristiche proprie.



Struttura principale della gerarchia

Partendo da sinistra:

- L'interfaccia `Set` **estende** (`is-a`) l'interfaccia `Collection`.
  Un `Set` è quindi una Collection con una regola fondamentale: non ammette duplicati.
- L'interfaccia `SortedSet` estende `Set`.
  Aggiunge la caratteristica dell'ordinamento automatico degli elementi secondo un criterio naturale o definito da un `Comparator`.

Al centro:

- L'interfaccia `List` estende `Collection`.
  Una `List` rappresenta **una sequenza ordinata di elementi**:
  - **mantiene l'ordine di inserimento**
  - **consente duplicati**
  - **permette accesso tramite indice**

A destra:

- L'interfaccia `Queue` estende `Collection`.
  Modella una struttura dati tipicamente utilizzata per gestire elementi secondo una politica:
  - **FIFO (First In First Out)**
  - **oppure altre varianti a seconda dell'implementazione**

L'albero mostrato nello schema **non rappresenta l'intero framework delle Collection**, ma solo la struttura principale delle interfacce.

Per ciascuna di queste sotto-interfacce esistono diverse **classi concrete** che ne forniscono l'implementazione.

Ad esempio:

- `List` → `ArrayList`, `LinkedList`
- `Set` → `HashSet`, `TreeSet`
- `Queue` → `LinkedList`, `PriorityQueue`

Questo significa che:

> **Le interfacce definiscono il *contratto* (cosa si può fare),**
> **le classi concrete definiscono il *comportamento reale* (come viene fatto).**

Interfaccia `List` (Interface `List`)

Java

```java
public interface List<E> extends Collection<E>
```

L'interfaccia `List` rappresenta:

- **una collezione sequenziale di oggetti, cioè una struttura dati in cui gli elementi vengono mantenuti in un ordine ben preciso.**

A differenza di altre Collection, una `List` è caratterizzata dal fatto che:

- **mantiene l'ordine di inserimento (o comunque un ordine definito)**
- **consente l'accesso diretto agli elementi tramite indice**
- **ammette duplicati**

Accesso tramite indice

Uno degli aspetti fondamentali della `List` è che:

- **ogni elemento occupa una posizione numerata (indice).**

Questo permette operazioni come:

```java
list.get(0);       // recupera il primo elemento
list.set(1, x);    // sostituisce l'elemento in posizione 1
list.add(2, y);    // inserisce un elemento in una posizione specifica
```

L'accesso indicizzato è ciò che distingue concettualmente una `List` da un `Set`, dove non esiste il concetto di posizione.

Duplicati

Una `List` può contenere più volte lo stesso elemento:

```java
list.add("A");
list.add("A");
```

Entrambi gli elementi verranno mantenuti.

Questo la rende adatta a rappresentare sequenze, cronologie, raccolte ordinate di dati, ecc.

Funzionalità aggiuntive rispetto a Collection

Oltre ai metodi ereditati da `Collection`, l'interfaccia `List` introduce operazioni specifiche per:

- i**nserimento in una posizione precisa**
- **ricerca dell'indice di un elemento (`indexOf`, `lastIndexOf`)**
- **sostituzione di elementi**
- **estrazione di sotto-liste (`subList`)**

Queste operazioni sono possibili proprio perché la `List` ha una struttura sequenziale e indicizzata.

`ListIterator`

Oltre all'`Iterator` standard, una `List` fornisce un iteratore più potente: `ListIterator`.

`ListIterator` è una specializzazione di `Iterator` che aggiunge funzionalità avanzate.

La differenza principale è che consente lo **scorrimento bidirezionale**:

- **avanti (`next()`)**
- **indietro (`previous()`)**

Inoltre permette:

- **di conoscere l'indice corrente**
- **di modificare elementi durante l'iterazione**
- **di inserire nuovi elementi durante la scansione**

Questo è possibile solo nelle `List`, perché solo esse hanno un ordine e un concetto di posizione.

Le `List`

Come abbiamo visto, `List` è un'interfaccia.
Per poterla utilizzare concretamente è necessario istanziare una classe che la implementi.

Le implementazioni più note dell'interfaccia `List` sono:

- `ArrayList<E>`
- `Vector<E>`
- `LinkedList<E>`

Ognuna di queste classi rispetta il contratto definito da `List`, **ma utilizza una struttura dati interna diversa, con conseguenze sulle prestazioni e sul comportamento**.

`ArrayList<E>`

È l'implementazione più utilizzata.

- **Internamente è basata su un array dinamico.**
- **Offre accesso rapido tramite indice.**

- **L’inserimento o la rimozione in mezzo alla lista può essere meno efficiente, perché richiede lo spostamento degli elementi**.
  Esempio Pratico con `ArrayList`

Java

```java
import java.util.ArrayList;
import java.util.List;

public class EsempioArrayList {

    public static void main(String[] args) {

        // Creazione di una ArrayList
        List<String> lista = new ArrayList<>();

        // 1️⃣ Inserimento elementi (array dinamico)
        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("D");

        System.out.println("Lista iniziale: " + lista);

        // 2️⃣ Accesso rapido tramite indice (O(1))
        String elemento = lista.get(2);   // accesso diretto alla posizione 2
        System.out.println("Elemento in posizione 2: " + elemento);

        // 3️⃣ Inserimento in posizione intermedia
        lista.add(1, "X");   // inserisce "X" in posizione 1

        System.out.println("Lista dopo inserimento in mezzo: " + lista);

        // 4️⃣ Rimozione in posizione intermedia
        lista.remove(2);

        System.out.println("Lista dopo rimozione: " + lista);
    }
}
```

È generalmente la scelta predefinita quando non ci sono esigenze particolari.

`Vector<E>`

- **Strutturalmente simile ad `ArrayList`.**
- **È sincronizzato, quindi thread-safe.**
- **Oggi è poco utilizzato perché il costo della sincronizzazione lo rende meno performante nella maggior parte dei casi.**

Esempio base di utilizzo di `Vector`

Java

```java
import java.util.Vector;

public class EsempioVector {

    public static void main(String[] args) {

        Vector<String> vector = new Vector<>();

        vector.add("A");
        vector.add("B");
        vector.add("C");

        System.out.println("Vector: " + vector);

        // Accesso tramite indice (come ArrayList)
        System.out.println("Elemento in posizione 1: " + vector.get(1));

        // Inserimento in posizione specifica
        vector.add(1, "X");

        System.out.println("Dopo inserimento: " + vector);
    }
}
```

Nota:

È considerato una classe “legacy”, mantenuta per compatibilità con versioni precedenti di Java.

`LinkedList<E>`

- **Internamente è basata su una lista doppiamente collegata.**
- **Non offre accesso rapido tramite indice.**
- **È più efficiente nelle operazioni di inserimento e rimozione frequenti.**

Può essere utilizzata anche come implementazione di `Queue`, grazie alla sua struttura.

Esempio base con `LinkedList`

Java

```java
import java.util.LinkedList;

public class EsempioLinkedList {

    public static void main(String[] args) {

        LinkedList<String> lista = new LinkedList<>();

        lista.add("A");
        lista.add("B");
        lista.add("C");

        System.out.println("Lista iniziale: " + lista);

        // Inserimento in testa (molto efficiente)
        lista.addFirst("START");

        // Inserimento in coda (molto efficiente)
        lista.addLast("END");

        System.out.println("Dopo inserimenti: " + lista);

        // Rimozione del primo elemento
        lista.removeFirst();

        System.out.println("Dopo rimozione: " + lista);
    }
```

```
}
```

Il ruolo del generico `<E>`

Il simbolo `<E>` rappresenta un tipo generico.

Indica il tipo di elemento che la lista conterrà.

Esempio:

```java
List<String> nomi = new ArrayList<>();
```

In questo caso:

- **`E` diventa `String`**
- **la lista potrà contenere solo oggetti di tipo `String`**
- **il compilatore garantisce la type safety**

L'uso dei generics evita il casting esplicito e riduce il rischio di errori a runtime.

Osservazione importante

È buona pratica dichiarare il tipo tramite l'interfaccia:

```java
List<String> lista = new ArrayList<>();
```

e non:

```java
ArrayList<String> lista = new ArrayList<>();
```

In questo modo il codice rimane più flessibile: si può cambiare implementazione senza modificare la logica del programma.

Classe `ArrayList`

`ArrayList` è una classe concreta che:

- **implementa l'interfaccia `List` utilizzando internamente un array a dimensione variabile**.

Questo significa che:

- **gli elementi sono memorizzati in un array**
- **quando lo spazio disponibile termina, l'array viene automaticamente ridimensionato**
- **l'accesso tramite indice è molto efficiente**

A differenza di un array tradizionale, quindi, **la dimensione non è fissa ma cresce dinamicamente in base alle necessità**.

Costruttori principali

1. `ArrayList()`

Java

```java
ArrayList<E> list = new ArrayList<>();
```

Costruisce una lista con capacità iniziale pari a 10.

> **Questo non significa che la lista contenga già 10 elementi, ma che l'array interno è inizialmente in grado di ospitarne fino a 10 prima di dover essere ridimensionato.**

2 `ArrayList(Collection<E> c)`

Java

```java
ArrayList<E> list = new ArrayList<>(collezione);
```

Costruisce una nuova `ArrayList` copiando gli elementi di una Collection esistente.

È utile quando si vuole:

- **creare una copia di una collezione**
- **convertire una Collection generica in una `ArrayList`**

La dimensione iniziale sarà pari al numero di elementi contenuti nella Collection passata.

3 `ArrayList(int initialCapacity)`

Java

```java
ArrayList<E> list = new ArrayList<>(50);
```

Costruisce una lista vuota specificando la capacità iniziale dell'array interno.

Questo è utile quando si conosce in anticipo il numero approssimativo di elementi da inserire, perché:

- **si evita il ridimensionamento automatico**
- **si migliora leggermente l'efficienza**

Capacita vs Dimensione

È importante distinguere tra:

- **Capacity (capacità)** → **dimensione dell'array interno**
- **Size (dimensione reale)** → **numero effettivo di elementi presenti nella lista**

Esempio:

Java

```java
ArrayList<String> list = new ArrayList<>(10);

```

- Capacity = 10
- Size = 0

Dopo:

Java

```java
list.add("A");

```

- Capacity = 10
- Size = 1

Come funziona il ridimensionamento?

Quando la capacità viene superata:

- **viene creato un nuovo array più grande**
- **gli elementi vengono copiati nel nuovo array**
- **il riferimento viene aggiornato**

Questa operazione ha un costo, motivo per cui specificare la capacità iniziale può essere utile in alcuni casi.

Metodi degli `ArrayList`

Come abbiamo detto, `ArrayList` è basato su un array dinamico, quindi molte operazioni implicano uno shift degli elementi.
A differenza dei classici Array che invece hanno una capacità fissa e immutabile.
Inoltre questa classe implementa diversi metodi molto utili per inserire, ricercare, leggere e rimuovere gli elementi.
Andiamo ad analizzare quelli più noti:

Metodi di inserimento degli `ArrayList`

1. `add(E element)`
   - **Aggiunge un elemento in fondo alla lista**

Java

```java
ArrayList<String> lista = new ArrayList<>();
lista.add("A");
lista.add("B");
```

Complessità media: O(1)

2. `add(int index, E element)`
   - Inserisce un elemento in una posizione specifica
   - Gli elementi successivi, con questo metodo, vengono spostati a destra (shift).

```java
lista.add(1, "X");
// Se lista = [A, B]
// Dopo → [A, X, B]
```

❌ **Eccezione:**

- `IndexOutOfBoundsException` se:
  - index < 0
  - index > size()

Complessità: O(n)

3. `addAll(Colletion<E> c)`
   - Aggiunge in fondo tutti gli elementi della Lista passata come argomento alla lista che chiamante

```java
ArrayList<String> altraLista = new ArrayList<>();
altraLista.add("C");
altraLista.add("D");

lista.addAll(altraLista);
```

4. `addAll(int index, Collection<E> c)`
   - Inserisce una collezione passata come argomento a partire da un indice, alla collezione chiamante

```java
ArrayList<String> altraLista = new ArrayList<>();
altraLista.add("C");
altraLista.add("D");

lista.addAll(1,altraLista);
```

- Anche qui avviene lo shift (spostamento a destra) degli elementi presenti nella collezione chiamante.

5. `set(int index, E element)`
    - Sostituisce un elemento senza effettuare lo shift degli elementi già presenti nella collezione

```java
lista.set(0, "Z");
```

Quindi:

- non cambia la dimensione dell' `ArrayList`
- Non sposta gli elementi già presenti

Eccezione

`IndexOutOfBoundsException` se index non valido

Metodi di lettura dell' `ArrayList`

1. `get(int index)`
    - Restituisce l'elemento presente alla posizione specifica (indice) passato come argomento della funzione

```java
String valore = lista.get(1);
```

Complessità: O(1)

(perché è accesso diretto su array)

Metodi di ricerca dell' `ArrayList`

1. `contains(E element)`
    - **Scorre tutta lista fino a che non trova l'elemento passato come argomento alla funzione**
    - Restituisce un booleano

```java
```

```java
boolean presente = lista.contains("A");
```

ⓘ Complessita: O(n)

2. `indexOf(E element)`
   - Restituisce l'indice della prima occorrenza dell'elemento passato come parametro alla funzione

Java

```java
int posizione = lista.indexOf("A");
```

Se non trova nessuna occorrenza ritorna `-1`.

3. `lastIndexOf(E element)`

- Restituisce l'indice dell'ultima occorrenza dell'elemento passato come argomento alla funzione

Java

```java
int posizione = lista.lastIndexOf("A");
```

Metodi di rimozione dell' `ArrayList`

1. `remove(int index)`
   - **Rimuove l'elemento alla posizione indicata.**

Java

```java
lista.remove(1);
```

- **Gli elementi successivi vengono spostati a sinistra.**
- **Restituisce l'elemento rimosso .**

Eccezione

Può lanciare `IndexOutOfBoundsException`.

2. `remove(E element)`

- **Rimuove la prima occorrenza.**

```java
boolean eliminato = lista.remove("A");
```

Restituisce:

- `true` **se trovato e rimosso**
- `false` **se non presente**

> **Anche qui avviene lo shift.**

3. `clear()`
   - **Svuota completamente la lista.**

```java
lista.clear();
```

4. `trimToSize()`
   - **Riduce la capacità interna dell'array alla dimensione attuale.**

```java
lista.trimToSize();
```

- **Utile per ottimizzare memoria dopo molte rimozioni.**

Altri metodi utili

1. `size()`
   - Restituisce un integer che rappresenta la lunghezza (non la capacità) dell' `ArrayList`

```java
int dimensione = lista.size();
```

2. `isEmpty()`
   - Restituisce un booleano e serve per vedere se la collezione è vuota

```java
boolean vuota = lista.isEmpty();
```

3. `toArray()`
   - Converte in array.

```java
Object[] array = lista.toArray();
```

Oppure versione tipizzata (più corretta):

```java
String[] array = lista.toArray(new String[0]);
```

Il metodo `.equals()` e confronto tra gli oggetti

Il metodo `.equals()` e confronto tra gli oggetti

Come visto nei metodi di ricerca e rimozione di `ArrayList`, operazioni come:

- `contains(obj)`
- `remove(obj)`
- `indexOf(obj)`
- `lastIndexOf(obj)`

non confrontano gli oggetti tramite `==`, ma utilizzano il metodo:

```java
equals(Object obj)
```

Questo significa che, quando la lista deve verificare se un elemento è presente o deve essere rimosso, non controlla se i due riferimenti puntano alla stessa area di memoria, ma invoca il metodo `equals()` per determinare se due oggetti sono logicamente equivalenti.

Remember

- `==` → **confronta gli indirizzi di memoria (identità dell'oggetto)**

- `equals()` → **confronta il contenuto logico dell'oggetto (uguaglianza semantica)**

Quando è necessario ridefinire `equals()`

Se gli elementi della lista sono oggetti appartenenti a una classe definita dal programmatore, allora è necessario ridefinire il metodo:

```java
public boolean equals(Object obj)
```

Infatti, la versione di default ereditata dalla classe `Object` confronta solo i riferimenti in memoria, cioè verifica se due variabili puntano **allo stesso oggetto**, esattamente come farebbe l'operatore `==`.

Questo significa che, senza override, due oggetti distinti ma con lo stesso contenuto verranno considerati diversi.

Ad esempio:

```java
Studente s1 = new Studente(123);
Studente s2 = new Studente(123);
```

Anche se rappresentano lo stesso studente (stessa matricola), sono stati creati con due `new` differenti, quindi occupano posizioni diverse in memoria.

Di conseguenza:

```java
s1 == s2           // false
s1.equals(s2)      // false (se equals non è ridefinito)
```

Di conseguenza, se inseriamo `s1` in un `ArrayList`

```java
lista.add(s1);
```

e poi eseguiamo:

```java
lista.contains(s2);
```

Il risultato sarà `false`, anche se logicamente rappresentano lo stesso studente.

Questo accade perché `ArrayList` utilizza `equals()` per verificare l'uguaglianza, e se `equals()` non è stato ridefinito, il confronto avviene solo sui riferimenti.

In conclusione, se non si ridefinisce `equals()`, metodi come `contains()` o `remove()` potrebbero non funzionare correttamente.

```java
equals(Object obj)
```

Questo significa che, per stabilire se due elementi sono "uguali", viene invocato il metodo `equals()` dell'oggetto.

Esempio:

```java
public class Studente {

    private int matricola;

    @Override
    public boolean equals(Object obj) {

        if (this == obj) return true;        // stesso oggetto
        if (obj == null) return false;      // confronto con null
        if (!(obj instanceof Studente)) return false;

        Studente other = (Studente) obj;
        return this.matricola == other.matricola;
    }
}
```

📋 Regole da rispettare quando si ridefinisce `equals()`

Quando si esegue l'override di `equals()`, non si sta semplicemente riscrivendo un metodo, ma si sta ridefinendo il concetto stesso di uguaglianza per quella classe.

Per questo motivo è necessario rispettare alcune regole fondamentali:

- **bisogna verificare che l'oggetto passato non sia `null`**
  Il metodo deve restituire `false` se l'oggetto di confronto è `null`, evitando errori e rispettando il contratto generale di `equals()`.
- **bisogna controllare il tipo (tramite `instanceof`)**
  Prima di effettuare il confronto, è necessario assicurarsi che l'oggetto sia compatibile con la classe corrente, così da evitare `ClassCastException` e garantire un confronto coerente.
- **bisogna confrontare gli attributi più significativi dell'oggetto**
  L'uguaglianza deve essere definita in base agli attributi che identificano logicamente l'oggetto (es. la matricola per uno studente), non su campi secondari o irrilevanti.
- **è buona pratica usare l'annotazione `@Override`**
  L'annotazione permette al compilatore di verificare che si stia effettivamente ridefinendo un metodo esistente, prevenendo errori di firma.

Contratto di equals

Oltre agli aspetti implementativi, il metodo `equals()` deve rispettare un vero e proprio contratto formale definito dalla classe `Object`.

Le proprietà fondamentali sono:

- **Riflessiva**
  `x.equals(x)` deve restituire `true`.
  Ogni oggetto deve essere uguale a sé stesso.
- **Simmetrica**
  Se `x.equals(y)` è `true`, allora anche `y.equals(x)` deve essere `true`.
  L'uguaglianza non può dipendere dall'ordine del confronto.
- **Transitiva**
  Se `x.equals(y)` è `true` e `y.equals(z)` è `true`, allora anche

`x.equals(z)` deve essere `true`.
L'uguaglianza deve essere logicamente coerente.

- **Consistente**
  Più invocazioni devono restituire lo stesso risultato, purché lo stato degli oggetti non cambi.
  Il risultato non deve essere casuale o dipendere da fattori esterni.
- **Confronto con null**
  Se `x` non è `null`, allora `x.equals(null)` deve restituire `false`.
  Nessun oggetto può essere uguale a `null`.

Il rispetto di queste proprietà garantisce che il comportamento delle collezioni e delle strutture dati sia corretto e prevedibile.

La relazione tra il metodo `equals()` e `hashCode()`

Quando si ridefinisce `equals()`, è necessario ridefinire anche `hashCode()`.

Questo perché il contratto generale del linguaggio Java stabilisce che:

> **Se due oggetti sono uguali secondo `equals()`, allora devono avere lo stesso valore di `hashCode()`.**

In altre parole, l'uguaglianza logica deve essere coerente con il valore hash restituito dall'oggetto.

Questo aspetto è fondamentale soprattutto per strutture basate su hashing, come `HashSet` e `HashMap`, che utilizzano `hashCode()` per determinare in quale "posizione" memorizzare un oggetto e `equals()` per verificare eventuali collisioni.

Se `equals()` viene ridefinito ma `hashCode()` no, si rischiano comportamenti incoerenti, come oggetti duplicati in un `HashSet` o chiavi non riconosciute in una `HashMap`.

Per questo motivo, `equals()` e `hashCode()` devono sempre essere considerati come una coppia inscindibile.

Classe `Vector<E>`

- Esiste dalla versione **1.0 di Java**, quindi è precedente all'introduzione del framework delle collezioni (Java 1.2).
- Funziona essenzialmente come un `ArrayList` (introdotto dalla versione 1.2).

- **Implementa l'interfaccia `List` utilizzando un array a dimensione variabile.**
- **A differenza di `ArrayList`, supporta la sincronizzazione.**

Dal punto di vista strutturale, `Vector` e `ArrayList` sono molto simili:

- **entrambi memorizzano gli elementi in un array che viene ridimensionato automaticamente quando la capacità non è più sufficiente.**

La differenza principale riguarda la **gestione della concorrenza**:

- **`Vector` è sincronizzato, quindi i suoi metodi sono thread-safe.**
- **`ArrayList` non è sincronizzato e quindi, in ambienti multi-thread, richiede gestione esterna della sincronizzazione.**

Questa sincronizzazione rende `Vector` generalmente **meno efficiente** rispetto ad `ArrayList` in contesti single-thread.

Costruttori

- Hanno la stessa firma e lo stesso funzionamento di quelli di `ArrayList` (ad esempio il costruttore senza parametri o con capacità iniziale).
- È disponibile inoltre il costruttore:

Java

```java
Vector(int initialCapacity, int capacityIncrement)
```

che costruisce un vettore vuoto specificando:

- `initialCapacity` → **la capacità iniziale dell'array**
- `capacityIncrement` → **l'incremento con cui la capacità viene aumentata quando l'array è pieno**

A differenza di `ArrayList`, che raddoppia automaticamente la capacità (o la aumenta secondo una politica interna), `Vector` **permette di controllare esplicitamente di quanto deve crescere l'array.**

Osservazione Didattica

Oggi `Vector` è considerata una classe legacy.
Nella maggior parte dei casi si preferisce usare `ArrayList`, **eventualmente**

combinata con meccanismi di sincronizzazione esterni, perché più flessibile ed efficiente.

Tuttavia è importante conoscerla per motivi storici e di compatibilità con codice meno recente.

Classe `LinkedList<E>`

A differenza di un `ArrayList`, che utilizza un array dinamico, `LinkedList` è composta da nodi collegati tra loro.
Ogni nodo contiene:

- il valore dell'elemento
- un riferimento al nodo successivo
- un riferimento al nodo precedente (lista doppiamente collegata).

Questo comporta alcune differenze importanti:

- **L'accesso per indice (`get(i)`) è meno efficiente rispetto ad `ArrayList`, perché richiede di attraversare la lista.**
- **Inserimenti e rimozioni all'inizio o alla fine sono molto efficienti, perché non richiedono lo spostamento degli elementi**.

Per questo motivo `LinkedList` è particolarmente adatta quando sono frequenti operazioni di inserimento e rimozione, soprattutto in testa o in coda.

Metodi specifici

Oltre ai metodi ereditati da `List`, `LinkedList` mette a disposizione operazioni tipiche delle strutture lineari come code e pile:

Java

```java
public void addFirst(E element)
public void addLast(E element)

public E getFirst()
public E getLast()

public E removeFirst()
public E removeLast()
```

Questi metodi permettono di:

- aggiungere un elemento in testa (`addFirst`)
- aggiungere un elemento in coda (`addLast`)
- leggere il primo o l'ultimo elemento senza rimuoverlo(`getFirst()` , `getLast()`).
- rimuovere direttamente il primo o l'ultimo elemento(`removeFirst()` , `removeLast()`).

La scelta tra le due classi dipende quindi dal tipo di operazioni predominanti nel problema da risolvere.

Grazie a questi metodi, `LinkedList` può essere utilizzata in modo naturale come:

- **coda (FIFO)** → `addLast()` + `removeFirst()`
- **pila (LIFO)** → `addFirst()` + `removeFirst()`

Confronto concettuale con `ArrayList`

- `ArrayList` → migliore per accesso rapido tramite indice
- `LinkedList` → migliore per inserimenti e rimozioni frequenti agli estremi

Collection e Generics

I **generics** sono una caratteristica presente in molti linguaggi di programmazione; in Java sono stati introdotti dalla versione **1.5**.

- **Consentono di creare collezioni o contenitori che memorizzano solo oggetti di un tipo specifico.**
  Il tipo viene indicato tra parentesi angolari `<>` .

Esempio generale:

Java

```java
CollectionType<Tipo> collection = new CollectionType<Tipo>();
```

Perche sono stati inrodotti i generics

Prima di Java 1.5, le collezioni memorizzavano oggetti di tipo `Object` .
Questo significava che:

- **era possibile inserire qualsiasi tipo di oggetto nella stessa collezione**
- **al momento dell'estrazione era necessario effettuare un cast esplicito**
- **eventuali errori di tipo venivano rilevati solo a runtime**

Con i generics, invece, il controllo del tipo viene effettuato **a compile-time**, cioè durante la compilazione.

Java

```java
ArrayList lista = new ArrayList();
lista.add("Mario");

String nome = (String) lista.get(0);  // cast obbligatorio
```

Problemi:

- Il cast è necessario.
- Se nella lista fosse stato inserito un oggetto non compatibile, si avrebbe una `ClassCastException` a runtime.
  Esempio con generics

Java

```java
ArrayList<String> lista = new ArrayList<String>();
lista.add("Mario");

String nome = lista.get(0);  // nessun cast necessario

```

In questo caso:

- **la lista può contenere solo `String`**
- **il compilatore impedisce l'inserimento di tipi incompatibili**
- **non è necessario alcun cast in fase di lettura**

Quindi grazie ai gnerics si hanno restrizioni sul tipo, ovvero:
supponiamo di avere un `ArrayList` di interi

```
ArrayList<Integer> numeri = new ArrayList<Integer>();
```

Grazie ai genrics sono sarà possibile fare:

```java
numeri.add("ciao");   // errore di compilazione
```

Poiché il vincolo è verificato dal compilatore, quindi l'errore viene intercettato prima dell'esecuzione del programma.

✓ Vantaggi dei genetics

- **Non sono più necessari cast per estrarre gli oggetti dalla collezione, se sono del tipo definito**
  Il tipo è già noto al compilatore.
- **Si evita l'eccezione di** `ClassCastException`
  Gli errori di tipo vengono rilevati a compile-time e non a runtime.
- Maggiore leggibilità del codice
  Il tipo di dati contenuto nella collezione è esplicitamente dichiarato.
- Maggiore sicurezza del tipo (type safety)
  Si riduce la possibilità di errori legati alla gestione di oggetti eterogenei.

Quindi i generics non modificano il funzionamento interno delle collezioni, ma introducono un **vincolo di tipo a livello di compilazione**.

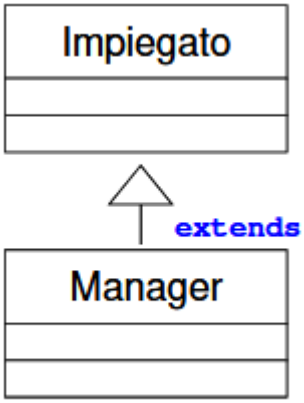
In sintesi:

- Prima → controllo dei tipi a runtime (meno sicuro)
- Dopo → controllo dei tipi a compile-time (più sicuro e pulito)

Per questo motivo oggi l'uso dei generics è considerato una pratica standard e imprescindibile quando si lavora con le collezioni in Java.

Esempio d'uso dei Generics con una gerarchia di classi

Supponiamo di avere una classe `Manager` che estende la classe padre `Impiegato`.



L'implementazione a nel codice è il seguente:

Java

```java
// creo un'ArrayList di Impiegati
ArrayList<Impiegato> elencoDipendenti =
    new ArrayList<Impiegato>();

// carico un impiegato e un manager: OK
elencoDipendenti.add(new Impiegato(...));
elencoDipendenti.add(new Manager(...));

// Non posso caricare oggetti non presenti nella
// gerarchia di Impiegato: ERRORE di compilazione
elencoDipendenti.add(new Date(...));

// recupero dell'impiegato SENZA cast
Impiegato imp = elencoDipendenti.get(0);

// recupero del manager CON cast
Manager mng = (Manager) elencoDipendenti.get(1);
```

Analisi del comportamento

In questo esempio viene dichiarata una collezione parametrizzata con il tipo `Impiegato`:

Java

```java
ArrayList<Impiegato>
```

Questo significa che la lista può contenere:

- **oggetti di tipo** `Impiegato`
- **oggetti di qualsiasi sottoclasse di** `Impiegato` **(es.** `Manager`**)**

Questo è permesso grazie al principio di **sostituibilità:**

- **una sottoclasse può essere trattata come la superclasse.**

Infatti, se `Manager` estende `Impiegato`, allora è perfettamente lecito scrivere:

```java
elencoDipendenti.add(new Manager(...));
```

perché vale il principio di sostituibilità (una sottoclasse può essere trattata come la superclasse).

Controllo dei tipi a compile-time

Se si prova a inserire un oggetto che **non appartiene alla gerarchia di** `Impiegato`, come:

```java
elencoDipendenti.add(new Date(...));
```

il compilatore genera un **errore**, impedendo l'inserimento di tipi incompatibili. **Questo garantisce la coerenza della collezione e riduce il rischio di errori in fase di esecuzione.**

Recupero degli elementi

Quando si estrae un elemento della lista:

```java
Impiegato imp = elencoDipendenti.get(0);
```

non è necessario alcun cast, perché il tipo della collezione è già noto.

Se invece si vuole trattare un elemento come una **sottoclasse**, ad esempio `Manager`, allora il cast diventa necessario:

```java
Manager mng = (Manager) elencoDipendenti.get(1);
```

Qui il cast è necessario perché:

- **Il cast serve perché il metodo `get()` restituisce un `Impiegato`, e il compilatore non può sapere con certezza se l'oggetto in quella posizione sia effettivamente un `Manager`.**

Il cast è quindi richiesto per specificare che si vuole trattare quell'oggetto come istanza della sottoclasse.

> **Osservazione importante**
>
> I generics non sono limitati ad `ArrayList`. Si applicano a tutte le strutture della gerarchia delle `Collection`, quindi:
>
> - liste (`List`)
> - insiemi (`Set`)
> - mappe (`Map`, dove si specificano chiave e valore)
>
> In ogni caso, **il tipo indicato tra `<>` definisce i vincoli sugli oggetti che la struttura può contenere, garantendo la sicurezza dei tipi già in fase di compilazione.**

Il costrutto `foreach`

Dalla JSE 5, tutte le **Collection parametrizzate con i generics** possono utilizzare una versione "evoluta" del classico `for`, nota come `foreach`.

La sintassi generale è la seguente:

Java

```java
for(Element element : collection) {
    // operazioni su element
}
```

Come funziona:

1. Si dichiara:
   - il tipo dell'elemento (`Element element`) — tipicamente corrisponde al tipo generico della collezione
   - la collezione su cui iterare (`collection`)
2. Non è necessario:
   - dichiarare e gestire manualmente un indice

- ottenere e usare esplicitamente un `Iterator`

> **Nota: Il `foreach` può essere usato anche con gli array, non solo con le Collection.**

> Il costrutto `foreach` è solo la forma compatta dell' `Iterator` per navigare una Collection

Esempio d'uso

Java

```java
LinkedList<Impiegato> lista = new LinkedList<>();

lista.add(new Impiegato("Mario", 1500, new Date()));
lista.add(new Impiegato("Gino", 1200, new Date()));
lista.add(new Impiegato("Luca", 1100, new Date()));

for(Impiegato dipendente : lista) {
    System.out.println("Dipendente: " + dipendente);
}
```

Cosa succede qui:

- **Ad ogni iterazione, la variabile `dipendente` assume uno alla volta i valori degli elementi presenti nella lista `lista`.**
- **Non serve gestire un indice né creare un `Iterator`.**
- **Il codice è più leggibile e riduce il rischio di errori comuni come l' `IndexOutOfBoundsException`.**

AutoBoxing & UnBoxing

Come abbiamo anticipato prima, in Java, **le Collection possono contenere solo oggetti, non tipi primitivi come `int`, `double`, `char` ecc.**

Questo può creare problemi quando vogliamo inserire direttamente valori primitivi in una lista o in un set, perché servirebbe sempre creare manualmente l'oggetto wrapper corrispondente (`Integer`, `Double`, `Character` ...).

Per ovviare a questo problema la Oracle ha introdotto i concetti di Autoboxing e Unboxing

1. **Autoboxing**
   - **Consente di convertire automaticamente un tipo primitivo nel corrispondente wrapper quando è necessario, ad esempio durante l'inserimento in una Collection.**

```java
Integer wrapper = 5; // int 5 viene “trasformato” in Integer
```

2. **Unboxing**
   - **Consente di estrarre automaticamente il valore primitivo da un oggetto wrapper, ad esempio quando vogliamo usarlo in operazioni aritmetiche o assegnarlo a una variabile primitiva.**

```java
int i = new Integer(5); // Integer viene convertito in int
```

In pratica, il compilatore gestisce automaticamente:

1. **La costruzione del wrapper durante l'autoboxing**
2. **Il cast automatico in primitivo durante l'unboxing**

Esempio pratico con le collection

```java
import java.util.*;

public class AutoBoxingExample {
    public static void main(String[] args) {
        // Lista di Double (wrapper), ma possiamo inserire valori primitivi
        List<Double> numeri = new LinkedList<>();

        // Autoboxing: i valori primitivi vengono convertiti automaticamente in Double
        numeri.add(1.5);
        numeri.add(3.61);
        numeri.add(12.722);

        // Unboxing: quando leggiamo i valori e li usiamo come primitivi
```

```java
        for(int i = 0; i < numeri.size(); i++) {
            double valore = numeri.get(i); // il wrapper Double viene convertito automaticamente in double
            System.out.println("Elemento " + i + ": " + valore);
        }
    }
}
```

Cosa succede:

- **L'inserimento di valori primitivi nella Collection viene automaticamente trasformato nei wrapper corrispondenti (`Double`, `Integer`, ecc.)**
- **All'estrazione, il valore wrapper viene convertito di nuovo nel tipo primitivo senza bisogno di cast espliciti**